

Multi-Tenant White-Label E-Commerce Platform

Complete Project Documentation

Version: 1.0

Date: July 27, 2026

Document Type: Architecture & Product Specification

Table of Contents

1. Executive Summary
 2. Requirement Analysis
 3. System Architecture Overview
 4. Portal Structure & User Roles
 5. Multi-Tenancy & Data Isolation
 6. Feature Entitlements & Screen Control
 7. RBAC — Role-Based Access Control
 8. Product Catalog Model
 9. White-Labeling & Branding
 10. Theme System
 11. Pluggable Feature Services
 12. Core Modules & Services
 13. Data Model
 14. API & Integration Layer
 15. Technology Stack Recommendations
 16. Deployment Architecture
 17. Security Requirements
 18. Phased Delivery Roadmap
 19. Business Model & Pricing Tiers
 20. Anti-Patterns & What to Avoid
 21. Appendix A — Feature Catalog
 22. Appendix B — Starter Themes
 23. Appendix C — Document Deliverables Checklist
-

1. Executive Summary

This document defines the architecture for a **multi-tenant, white-label e-commerce SaaS platform** — a single commerce engine that powers many independent brands (tenants/enterprises). Each brand receives its own storefront, admin panel, users, products, orders, and branding configuration, with strict data isolation from other brands.

The platform owner operates a **Super Admin Console** to onboard enterprises, manage billing, control feature access, publish themes, register pluggable feature services, and perform platform-wide configuration.

Key Capabilities

- **Multi-tenant SaaS** — One engine, many independent brands
 - **Complete white-labeling** — Logos, colors, fonts, custom domains, email branding
 - **Flexible catalog** — Clothing, jewelry, electronics, or any vertical via attribute-based products
 - **Super Admin feature control** — Enable/disable modules and screens per tenant
 - **Theme system** — Pluggable storefront themes, easy to add new designs
 - **Extensible architecture** — Optional feature services (Virtual Checkout, Outfit Combiner) integrate without modifying core commerce logic
 - **Enterprise RBAC** — Tenant-scoped roles with permission and data scoping
-

2. Requirement Analysis

2.1 Original Requirements (Validated)

Requirement	Industry Term	Status
Main engine only platform owner accesses	Platform / Super Admin	Correct
Admin panel per brand	Tenant Admin	Correct
User-facing panel with branding	Storefront + Customer accounts	Correct
Logos, colors, brand name customization	White-label / Theming	Correct
Different product types (clothes, jewelry, electronics)	Flexible catalog model	Correct — use attributes
Role control per enterprise	Tenant-scoped RBAC	Correct
Super Admin controls which features/screens tenants get	Feature Entitlements	Correct — essential
Add themes easily	Theme Registry + Plugin model	Correct
Add features later as separate services	Pluggable Feature Services	Correct

2.2 Refinements

Split "User Panel" into Two Layers

Layer	Users	Purpose
Storefront	End customers (shoppers)	Browse, cart, checkout, order history
Tenant Staff Portal	Enterprise employees	Manage products, orders, customers with assigned roles

Do not combine customer login and staff login in one panel — they have different permissions, UX, and security boundaries.

Unified RBAC (Not Separate Menu/Data Roles)

Use one system with:

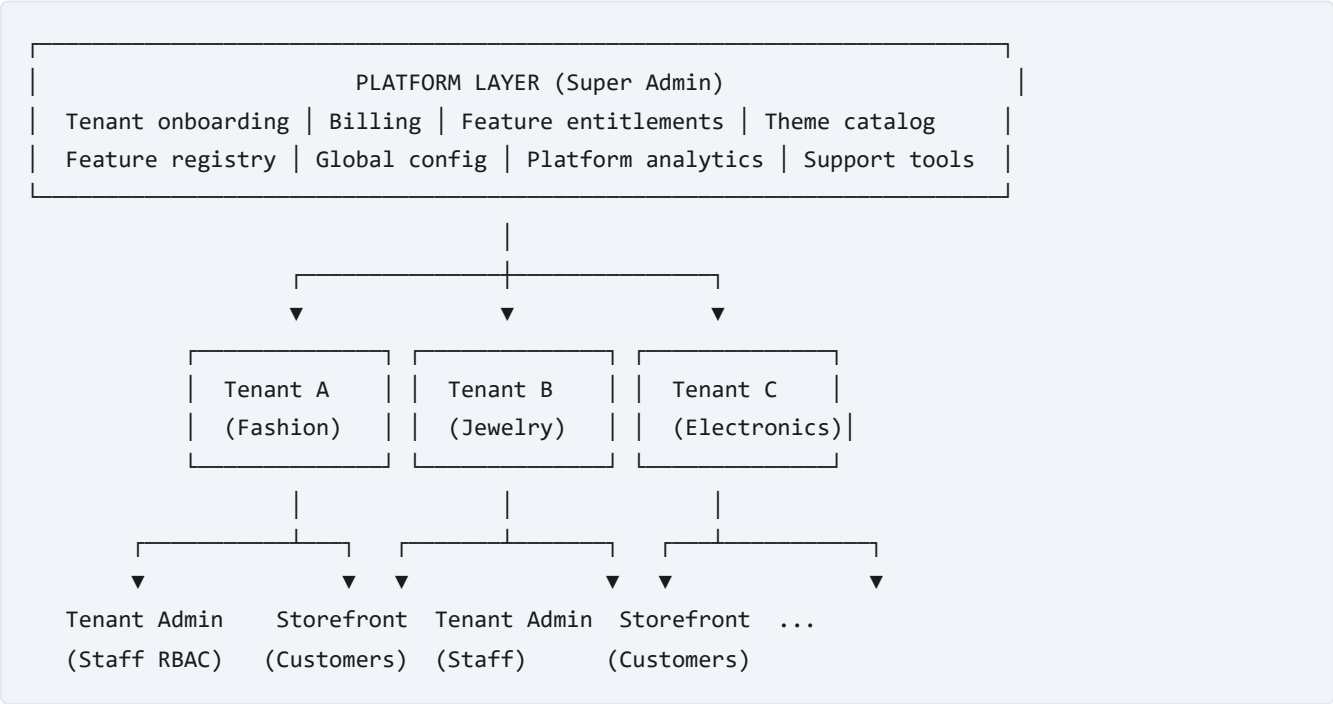
- **Permissions** — What actions are allowed (`products.create` , `orders.refund`)
- **Scopes** — What data they apply to (`tenant` , `store` , `own_records_only`)
- **Roles** — Bundles of permissions (`Tenant Owner` , `Catalog Manager`)

Menu visibility is derived from permissions, not configured separately.

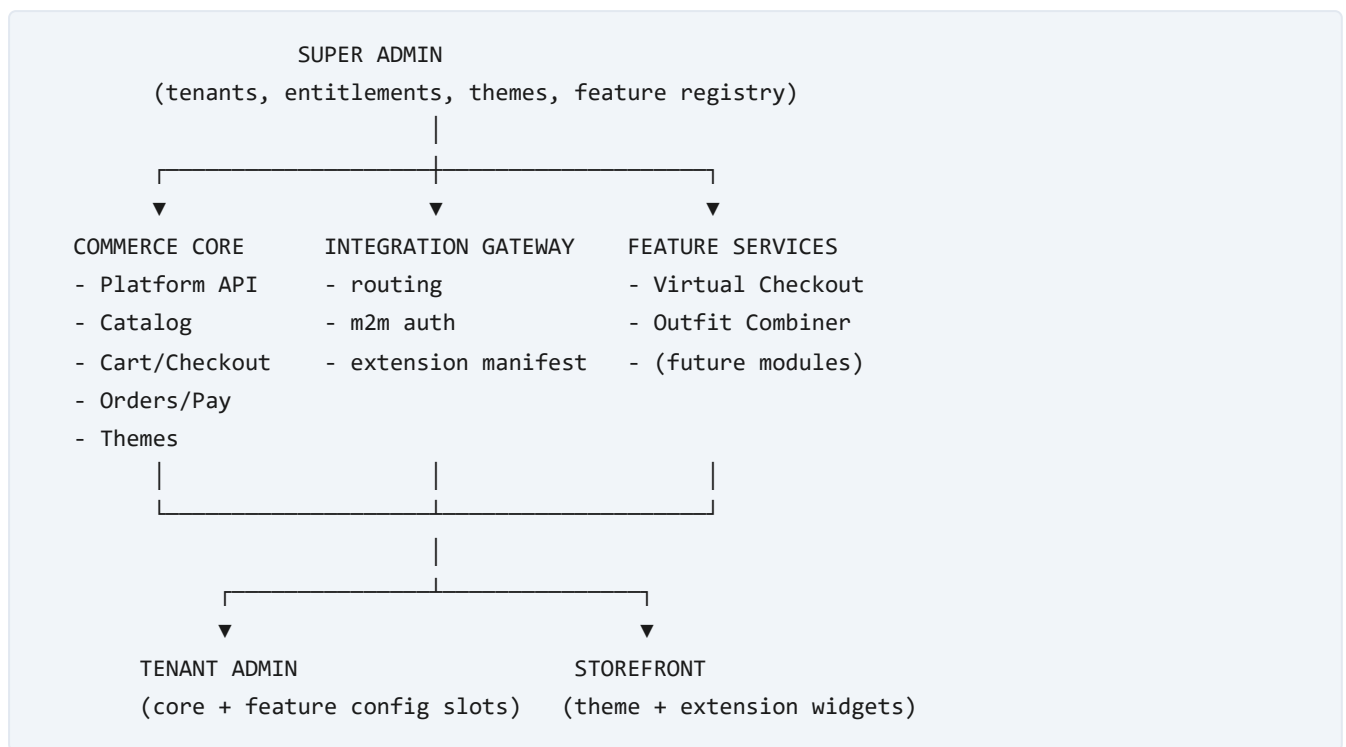
Tiered Customization

Tier	Customizable	Fixed by Platform
Branding	Logo, colors, fonts, favicon, email templates	Core layout components
Content	Pages, banners, navigation, footer	Checkout flow structure
Catalog	Categories, attributes, variants	Product data model
Domain	Custom domain (<code>shop.brand.com</code>)	SSL, CDN
Advanced	Custom CSS, page builder blocks	Payment/security core

3. System Architecture Overview



Integration with Feature Services



4. Portal Structure & User Roles

4.1 Three Portals

Portal	Access	Primary Functions
Super Admin Console	Platform team only	Tenants, plans, entitlements, themes, feature registry, billing, support
Tenant Admin	Enterprise staff	Catalog, orders, customers, branding, theme selection, staff management
Storefront	End customers	Shopping, cart, checkout, account

4.2 Role Hierarchy

Platform Level:

- Platform Super Admin — Full access
- Platform Support — Read-only + audited impersonation
- Platform Billing Admin — Subscriptions only

Tenant Level:

- Tenant Owner — All tenant permissions (within entitlements)
 - Store Admin — Everything except billing/delete tenant
 - Catalog Manager — Products, categories, inventory
 - Order Manager — Orders, refunds, shipping
 - Marketing — CMS, banners, coupons
 - Support Agent — Read orders/customers, limited edits
 - Viewer — Read-only reports
-

5. Multi-Tenancy & Data Isolation

5.1 Tenancy Models

Model	Best For	Tradeoff
Shared DB + <code>tenant_id</code>	Most SaaS (recommended start)	Must enforce tenant filter everywhere
Schema per tenant	Strong isolation, moderate scale	Migration complexity
DB per tenant	Enterprise/regulated clients	Highest ops cost

Recommendation: Start with shared database + strict `tenant_id` + PostgreSQL Row-Level Security. Migrate high-value tenants to dedicated schema/DB later if needed.

5.2 Domain Routing

Method	Use Case
Subdomain	MVP/dev (<code>fashion.platform.com</code>)
Custom domain	Production white-label (<code>shop.clientbrand.com</code>)
Path-based	Not recommended for white-label

5.3 Isolation Rules

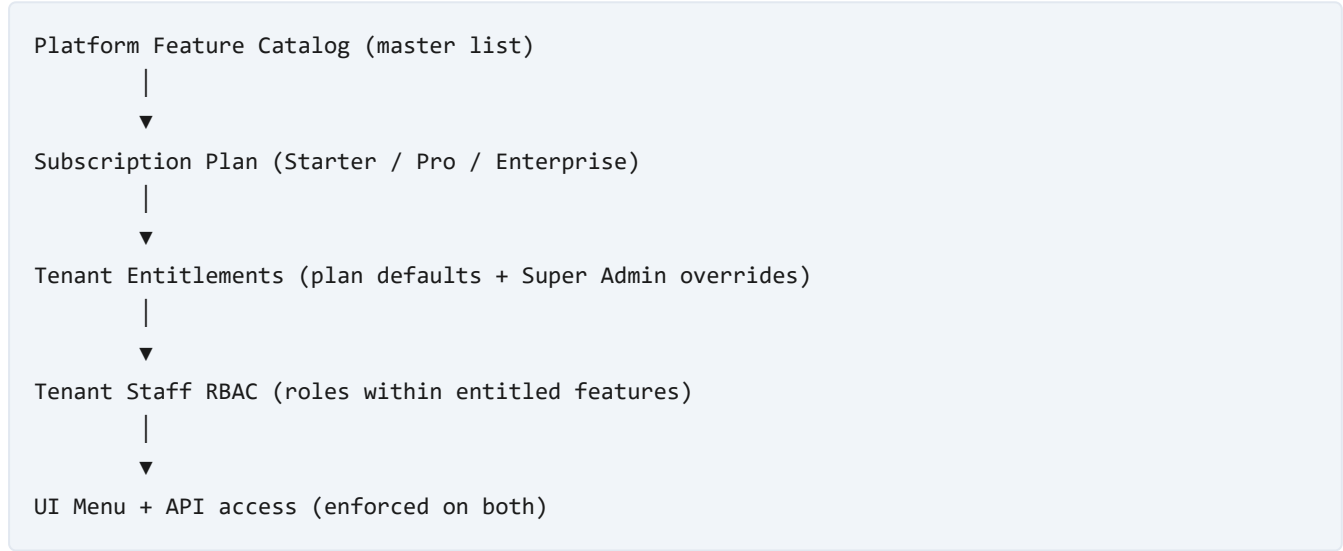
1. Every API query filters by `tenant_id` from JWT/session — never from client input alone
2. Middleware resolves tenant from domain → attaches to request context
3. Database indexes on `(tenant_id, id)` ; consider PostgreSQL RLS

- 4. File storage paths: `/tenants/{tenant_id}/media/...`
- 5. Separate auth realms for platform, tenant staff, and customers
- 6. Audit logs for admin actions and impersonation
- 7. Rate limits per tenant

6. Feature Entitlements & Screen Control

Super Admin controls **what each tenant is allowed to use** — modules, screens, and limits. This is Layer 1 of access control (Layer 2 is tenant staff RBAC).

6.1 Hierarchy



6.2 Building Blocks

Building Block	Purpose	Example
Feature	Capability or module	<code>marketing.coupons</code> , <code>reports.advanced</code>
Screen/Route	UI page tied to features	<code>/admin/marketing/coupons</code>
Entitlement	Tenant ON/OFF for a feature	Tenant A: coupons OFF

Rule: Feature OFF → screens hidden AND APIs return `403 Forbidden`.

6.3 Control Levels

Module-level (default): Toggle entire sections (Marketing, Reports, etc.)

Screen-level (overrides): Toggle individual pages within an enabled module

Limits: Max products, max staff, storage MB, API rate limits

6.4 Super Admin Tenant Features Panel

Super Admin can per tenant:

- Enable/disable modules and screens
- Override plan defaults
- Set resource limits
- Reset to plan defaults
- Clone entitlements from another tenant
- Preview effective menu as tenant admin
- Audit all entitlement changes

6.5 Enforcement

Frontend: menu = entitled features n staff permissions

Backend: `@RequireFeatures('marketing.coupons')` on every protected route

Frontend hiding is UX only — backend must always enforce entitlements.

7. RBAC — Role-Based Access Control

7.1 Two-Layer Model

Question	Answered By
Can this tenant use Coupons at all?	Super Admin entitlements
Can this staff user create coupons?	Tenant RBAC (<code>coupons.write</code>)
Can staff see Coupons menu?	Both: feature ON AND permission granted

7.2 Data Scoping Examples

Permission	Scope Rule
orders.view	All tenant orders OR assigned_to_me only
products.edit	All products OR category subtree only
reports.view	Store/region filter

7.3 Bootstrap Payload (Tenant Admin Login)

```
{
  "tenant": { "id": "uuid", "name": "LuxJewels", "plan": "professional" },
  "entitlements": ["catalog.products", "orders.refund", "marketing.coupons"],
  "limits": { "max_products": 5000, "current_products": 1203 },
  "screens": ["products.list", "coupons.list"],
  "permissions": ["products.read", "coupons.write"]
}
```

8. Product Catalog Model

Use **category + attribute schema** — one engine for all verticals.

8.1 Structure

Product

- base: name, slug, description, status, media
- category_id → defines applicable attributes
- variants[]
 - sku, price, compare_at_price
 - attribute_values: { "size": "M", "color": "Red" }

AttributeDefinition (per tenant or platform template)

- name: "Ring Size"
- type: select | text | number | boolean
- applies_to_categories: ["rings", "bracelets"]
- options: ["5", "6", "7"]

8.2 Industry Templates (Onboarding)

Template	Pre-loaded Categories	Sample Attributes
Fashion	Tops, Bottoms, Dresses	Size, Color, Material, Fit
Jewelry	Rings, Necklaces, Bracelets	Metal, Stone, Carat, Ring Size
Electronics	Phones, Laptops, Accessories	Storage, RAM, Color, Warranty

Templates are fully editable after onboarding.

9. White-Labeling & Branding

9.1 Three Layers

Layer	Controller	Examples
Platform Design System	Platform team	Buttons, forms, cart logic, checkout
Theme	Super Admin publishes; Tenant selects	Layouts, sections, component variants
Tenant Branding	Tenant Admin	Logo, colors, fonts, custom CSS

9.2 ThemeConfig Example

```
{
  "tenant_id": "uuid",
  "brand_name": "Luxe Gems",
  "logo_url": "https://cdn.../logo.svg",
  "favicon_url": "https://cdn.../favicon.ico",
  "primary_color": "#1a1a2e",
  "secondary_color": "#e94560",
  "font_heading": "Playfair Display",
  "font_body": "Inter",
  "custom_css": ".header { ... }",
  "layout_template": "jewelry-minimal-v2"
}
```

9.3 Design Tokens (CSS Variables)

```
:root {  
  --color-primary: #C9A227;  
  --color-background: #FAFAFA;  
  --font-heading: 'Playfair Display', serif;  
  --font-body: 'Inter', sans-serif;  
  --radius-card: 4px;  
}
```

Token Source	Examples
Theme	Card radius, header height, section spacing
Tenant branding	Primary color, logo URL, fonts
Tenant theme settings	Hero style, product card variant

10. Theme System

10.1 Theme as Plugin Package

Each theme is a self-contained package:

```
platform/themes/  
├─ registry.json  
├─ luxury-minimal/  
│   ├─ theme.json  
│   ├─ preview.png  
│   ├─ tokens.default.json  
│   ├─ layouts/  
│   ├─ sections/  
│   └─ components/  
├─ fashion-bold/  
└─ jewelry-classic/
```

10.2 Theme Manifest

```
{
  "id": "luxury-minimal",
  "name": "Luxury Minimal",
  "version": "1.2.0",
  "industry_tags": ["jewelry", "fashion", "beauty"],
  "required_features": ["catalog.products"],
  "layouts": ["home", "collection", "product", "cart", "page"],
  "sections": ["hero-banner", "featured-collection", "newsletter"],
  "settings_schema": {
    "hero_style": { "type": "select", "options": ["full-bleed", "split"] },
    "product_card_style": { "type": "select", "options": ["minimal", "overlay"] }
  }
}
```

10.3 Adding a New Theme (Developer Workflow)

1. Scaffold: `npm run theme:create jewelry-classic`
2. Implement required layouts: Home, Collection, Product, Cart, ContentPage
3. Add 3–5 sections: hero, featured-collection, image-with-text, newsletter
4. Implement component variants: ProductCard, Header, Footer
5. Capture preview screenshot
6. Register in theme registry; publish in Super Admin
7. Assign to plans / industry templates

No core API or checkout code changes required.

10.4 Page Builder (Tenant Config)

```
{
  "tenant_id": "uuid",
  "theme_id": "luxury-minimal",
  "pages": {
    "home": {
      "sections": [
        { "type": "hero-banner", "settings": { "title": "Summer Collection" } },
        { "type": "featured-collection", "settings": { "collection_id": "..." } }
      ]
    }
  }
}
```

10.5 Super Admin Theme Controls

- Publish/unpublish themes platform-wide
- Per-tenant allowed themes list
- Default theme for new tenants (by industry)
- Theme versioning and tenant pinning
- Entitlement keys: `themes.basic` , `themes.full_catalog` , `branding.custom_css`

11. Pluggable Feature Services

Optional features built as **separate services** that integrate via a stable contract — enabling Virtual Checkout, Outfit Combiner, and future modules without modifying commerce core.

11.1 Commerce OS + Feature Apps Model

Part	Responsibility
Commerce Core	Multi-tenant shop engine (catalog, cart, orders, payments, themes)
Feature Services	Specialized experiences (AI, 3D, virtual checkout)
Integration Layer	APIs, events, entitlements, UI slots

Rule: Feature services never own orders, tenants, or payments directly.

11.2 Four Integration Hooks

- 1. **Feature Registration** — Manifest in Super Admin feature registry
- 2. **API Contract** — Platform API (core → features) + Feature API (features → UI)
- 3. **Events** — Async pub/sub (`product.updated` , `order.created`)
- 4. **UI Extension Points** — Storefront and admin slots for widgets

11.3 UI Extension Slots

Storefront:

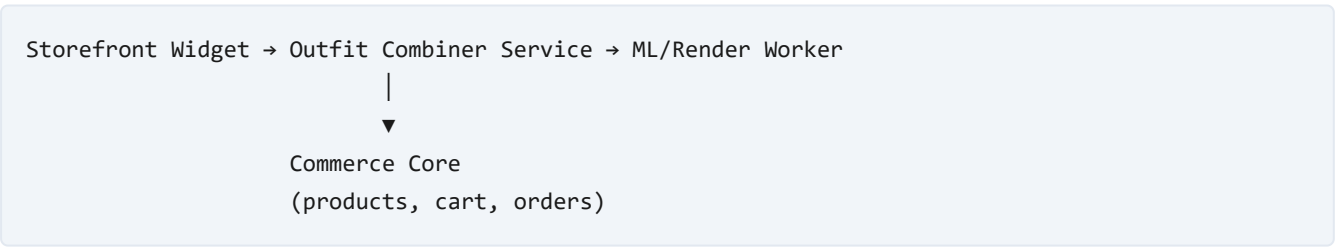
- `product.detail.actions` — "Try combinations", "Virtual try-on"
- `product.detail.gallery`
- `checkout.alternatives` — Virtual checkout
- `cart.summary.footer`

Tenant Admin:

- `catalog.product.sidebar`
- `settings.integrations`

11.4 Example: Outfit Combiner Service

User flow: Pick shirt + pants → see combinations on model → pick best look → add to cart.



In Core	In Outfit Combiner Service
Products, variants, images	Combination sessions
Categories (shirts, pants)	Render jobs, cached previews
Cart / orders	Model assets, pose templates

Core handles money path; feature service handles heavy rendering.

11.5 Example: Virtual Checkout Service

- Feature owns: UX flow, session state, optional AI assistant

- Core owns: Payment capture, order record, inventory deduction
- On complete: Feature calls `POST /platform/orders`

11.6 Feature Manifest

```
{
  "id": "outfit-combiner",
  "name": "Outfit Combination Studio",
  "entitlement_key": "features.outfit_combiner",
  "required_core_features": ["catalog.products", "storefront.cart"],
  "platform_api_scopes": ["products.read", "cart.write", "tenant.read"],
  "events_publish": ["outfit.session.completed"],
  "events_subscribe": ["product.updated"],
  "storefront_extensions": [
    { "slot": "product.detail.actions", "bundle": "/widgets/outfit-combiner.js" }
  ]
}
```

11.7 New Feature Checklist

1. Create service repo with manifest
 2. Implement Platform SDK calls (never direct DB access to core)
 3. Build storefront widget (Web Component or Module Federation)
 4. Register in Feature Registry
 5. Add entitlement key + plan mapping
 6. Enable for pilot tenant
-

12. Core Modules & Services

Service	Responsibility
Identity & Auth	Platform users, tenant staff, customers — separate realms
Tenant Resolver	Domain/subdomain → tenant context
Catalog Service	Products, variants, attributes, categories
Inventory Service	Stock per location/warehouse
Cart & Checkout	Sessions, pricing, tax, shipping
Order Service	Order lifecycle, fulfillment
Payment Service	Stripe/PayPal adapters, webhooks
Notification Service	Email/SMS with tenant-branded templates
Media Service	Tenant-scoped file storage
Theme Service	Brand tokens, layout config, theme registry
Entitlement Service	Feature/screen resolution per tenant
Feature Registry	Pluggable service catalog
Audit Log	Admin action tracking

Super Admin Modules

- Tenant lifecycle (create, suspend, delete, plan assignment)
- Feature entitlements and screen overrides
- Theme catalog management
- Feature service registry
- Global payment/email/CDN configuration
- Platform analytics (MRR, GMV, active tenants)
- Audited impersonation/support mode

Tenant Admin Modules

- Branding and theme selection
- Catalog, inventory, orders, customers

- CMS pages and navigation
- Staff and role management
- Shipping zones, tax rules, payment methods
- Feature-specific configuration (when entitled)
- Reports and analytics

Storefront Modules

- Product listing, search, filters
 - Product detail, reviews, wishlist
 - Cart, checkout, payments
 - Customer account
 - Theme-rendered UI + entitled feature widgets
-

13. Data Model

13.1 Platform Entities

```
PlatformUser
Tenant
SubscriptionPlan
TenantSubscription
Feature (catalog)
PlanFeature
Theme
FeatureService (registry)
PlatformAuditLog
```

13.2 Tenant-Scoped Entities (all include tenant_id)

```
TenantUser
Role, Permission
Customer
Product, ProductVariant
Category, AttributeDefinition
InventoryItem
Order, OrderLine, Payment, Shipment
Cart, CartItem
ThemeConfig, TenantPageSections
ContentPage, Menu, MenuItem
TenantEntitlement, TenantFeatureLimit
ShippingZone, TaxRule
Webhook, Integration
```

13.3 Entitlement Tables

```
features          -- master catalog
plans             -- starter, pro, enterprise
plan_features     -- plan_id + feature_key + enabled + limit
tenant_entitlements -- tenant_id + feature_key + enabled + source
tenant_feature_limits -- tenant_id + limit_key + value
tenant_screen_overrides -- optional screen-level control
```

14. API & Integration Layer

14.1 Platform API (Core exposes to Feature Services)

```
GET  /platform/tenants/{id}/context
GET  /platform/products/{id}
GET  /platform/carts/{id}
POST /platform/orders
POST /platform/carts/{id}/items
```

14.2 Storefront BFF Gateway

`https://shop.tenant.com/api/*`

|



Storefront BFF

- tenant resolve
- entitlement check
- route /core/* → commerce-core
- route /features/* → feature services

14.3 Storefront Extensions Endpoint

GET /storefront/extensions?tenant_id=...

→ Returns enabled widgets (URLs, slots, config)

14.4 Events

Core publishes: `product.updated`, `cart.item_added`, `order.created`, `tenant.settings_changed`

Features publish: `outfit.session.completed`, `virtual_checkout.session.completed`

15. Technology Stack Recommendations

Recommended Stack

Layer	Technology
Backend API	Node.js (NestJS) or Python (Django/FastAPI)
Database	PostgreSQL
Cache/Queue	Redis + BullMQ
Search	Meilisearch
Storage	S3-compatible (AWS S3, Cloudflare R2)
Frontend Admin	React + Next.js
Storefront	Next.js (SSR for SEO) with theme engine
Auth	Auth0/Clerk or Keycloak (multi-realm)
Payments	Stripe Connect
Email	Resend/SendGrid
Feature UI	Web Components or Module Federation

Project Structure (Monorepo)

```
apps/  
  commerce-core/           -- platform-api, super-admin, tenant-admin, storefront  
  integration-gateway/  
packages/  
  commerce-core-lib/  
  design-system/  
  theme-contract/  
  platform-sdk/  
  theme-luxury-minimal/  
  theme-fashion-bold/  
feature-services/          -- separate deploy units  
  virtual-checkout/  
  outfit-combiner/  
shared/  
  event-schemas/  
  extension-contract/
```

16. Deployment Architecture



- Single codebase, multi-tenant (not one deploy per client)
- Environments: dev → staging → production
- CI/CD with tenant isolation test suite

17. Security Requirements

Area	Requirement
Tenant isolation	Zero cross-tenant data leaks (automated tests)
API enforcement	Entitlements + RBAC on every protected route
Auth separation	Platform, tenant staff, customer — separate issuers
File storage	Tenant-scoped paths
Feature services	m2m JWT + tenant context validation
Audit	All Super Admin entitlement changes logged
GDPR	Export/delete customer data per tenant
Availability	99.9% uptime target
Backups	Daily DB backups, point-in-time recovery

18. Phased Delivery Roadmap

Phase 1 — Foundation (8–12 weeks)

- Tenant model + Super Admin (CRUD tenants)
- Tenant Admin auth + basic RBAC
- Product/catalog (variants, categories)
- Basic storefront + cart/checkout (Stripe)
- ThemeConfig: logo, colors, brand name
- Subdomain routing
- Feature registry tables (stub)
- 2 default themes

Exit: 2 demo tenants with different branding selling products.

Phase 2 — White-Label & Enterprise (6–8 weeks)

- Custom domains + SSL automation
- Attribute system + industry templates
- CMS pages + menu builder
- Email branding
- Super Admin feature entitlements (module-level)
- Advanced RBAC + audit logs
- Inventory, shipping zones, tax
- Theme registry + 3 additional themes
- Page builder (section reorder)

Phase 3 — Extensibility & Monetization (6–8 weeks)

- Platform API + platform-sdk
- Integration gateway + UI extension slots
- Subscription billing (plans, limits)
- Stripe Connect
- First pluggable feature: Outfit Combiner MVP
- Analytics dashboards
- Webhooks & public API
- Screen-level entitlement overrides

Phase 4 — Advanced (Ongoing)

- Virtual Checkout service
 - AI rendering for outfit combinations
 - Theme versioning + migrations
 - Private/custom themes for enterprise
 - Multi-store per tenant (regions/currencies)
 - B2B features (wholesale pricing)
 - Feature marketplace (optional)
-

19. Business Model & Pricing Tiers

Plan	Features
Starter	Subdomain, 2 themes, 100 products, 2 staff, basic modules
Professional	Custom domain, all standard themes, attributes, CMS, 10 staff, coupons
Enterprise	Custom CSS, API access, feature services, dedicated support, SLA, optional isolated DB

Revenue: Monthly subscription + optional transaction fee (Stripe Connect application fee).

Add-ons: Outfit Combiner, Virtual Checkout, advanced analytics, extra storage.

20. Anti-Patterns & What to Avoid

1. Do not deploy a separate codebase per brand
 2. Do not store tenant_id only in frontend
 3. Do not merge staff and customer auth
 4. Do not hardcode product types — use attributes
 5. Do not build payment processing from scratch
 6. Do not promise infinite customization on day one — tier it
 7. Do not hide menus without API enforcement
 8. Do not let feature services write directly to core database
 9. Do not allow tenant-uploaded HTML themes without sandboxing
 10. Do not duplicate feature lists in frontend and backend
-

Appendix A — Feature Catalog

Category	Feature Key	Description
Catalog	catalog.products	Products & variants
Catalog	catalog.categories	Category management
Catalog	catalog.attributes	Custom attributes
Catalog	catalog.bulk_import	Bulk product import
Inventory	inventory.tracking	Stock tracking
Inventory	inventory.multi_warehouse	Multi-warehouse
Orders	orders.management	Order management
Orders	orders.refunds	Refunds
Customers	customers.crm	Customer management
Marketing	marketing.coupons	Discount coupons
Marketing	marketing.abandoned_cart	Abandoned cart emails
Content	content.cms	CMS pages
Content	content.page_builder	Homepage builder
Storefront	storefront.reviews	Product reviews
Storefront	storefront.wishlist	Wishlist
Reports	reports.basic	Basic analytics
Reports	reports.advanced	Advanced analytics
Integrations	integrations.api	REST API access
Integrations	integrations.webhooks	Webhooks
Branding	branding.custom_domain	Custom domain
Branding	branding.custom_css	Custom CSS
Themes	themes.basic	2 default themes
Themes	themes.full_catalog	All published themes

Features	features.outfit_combiner	Outfit Combination Studio
Features	features.virtual_checkout	Virtual Checkout

Appendix B — Starter Themes

Theme ID	Name	Industry Tags	Key Sections
default	Default Store	general	hero, featured-products, newsletter
fashion-bold	Fashion Bold	fashion, apparel	hero-video, lookbook, featured-collection
luxury-minimal	Luxury Minimal	jewelry, beauty, fashion	hero-split, image-with-text, testimonials
electronics-clean	Electronics Clean	electronics, tech	spec-highlights, category-grid, deals
jewelry-classic	Jewelry Classic	jewelry	full-bleed-hero, collection-showcase, story

Appendix C — Document Deliverables Checklist

When implementation begins, produce:

1. **PRD** — User stories per persona
2. **Technical Architecture Document** — This document + API contracts
3. **Data Dictionary** — All entities and relationships
4. **RBAC Matrix** — Roles × permissions spreadsheet
5. **Feature Entitlement Matrix** — Plans × features spreadsheet
6. **Theme System Spec** — Tokens, templates, section types
7. **Feature Service Integration Spec** — Manifest schema, Platform API OpenAPI
8. **Tenant Onboarding Flow** — Signup → template → domain → go-live
9. **Security & Compliance Doc** — Isolation, audit, GDPR
10. **UI Wireframes** — Super Admin, Tenant Admin, Storefront
11. **Test Plan** — Cross-tenant isolation + entitlement enforcement tests

End of Document

Multi-Tenant White-Label E-Commerce Platform — Architecture & Product Specification v1.0